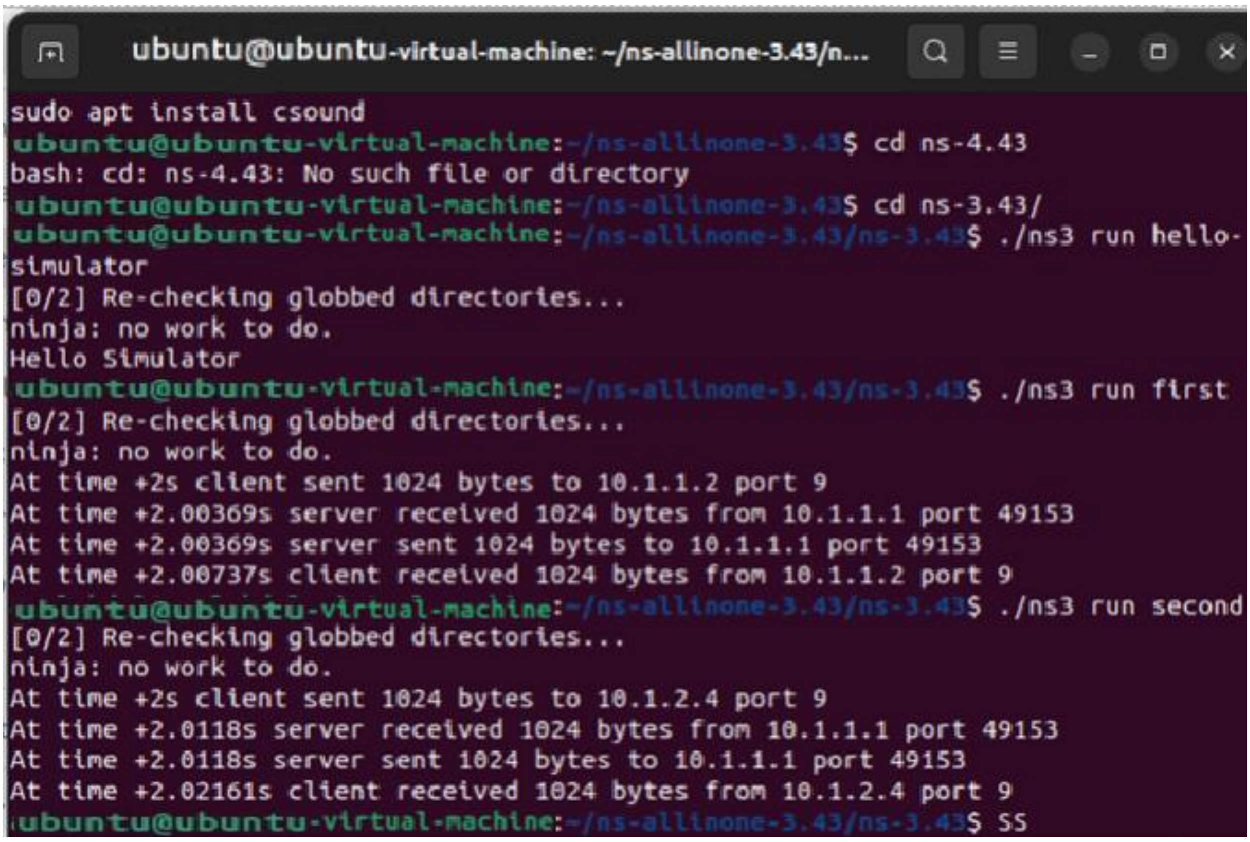
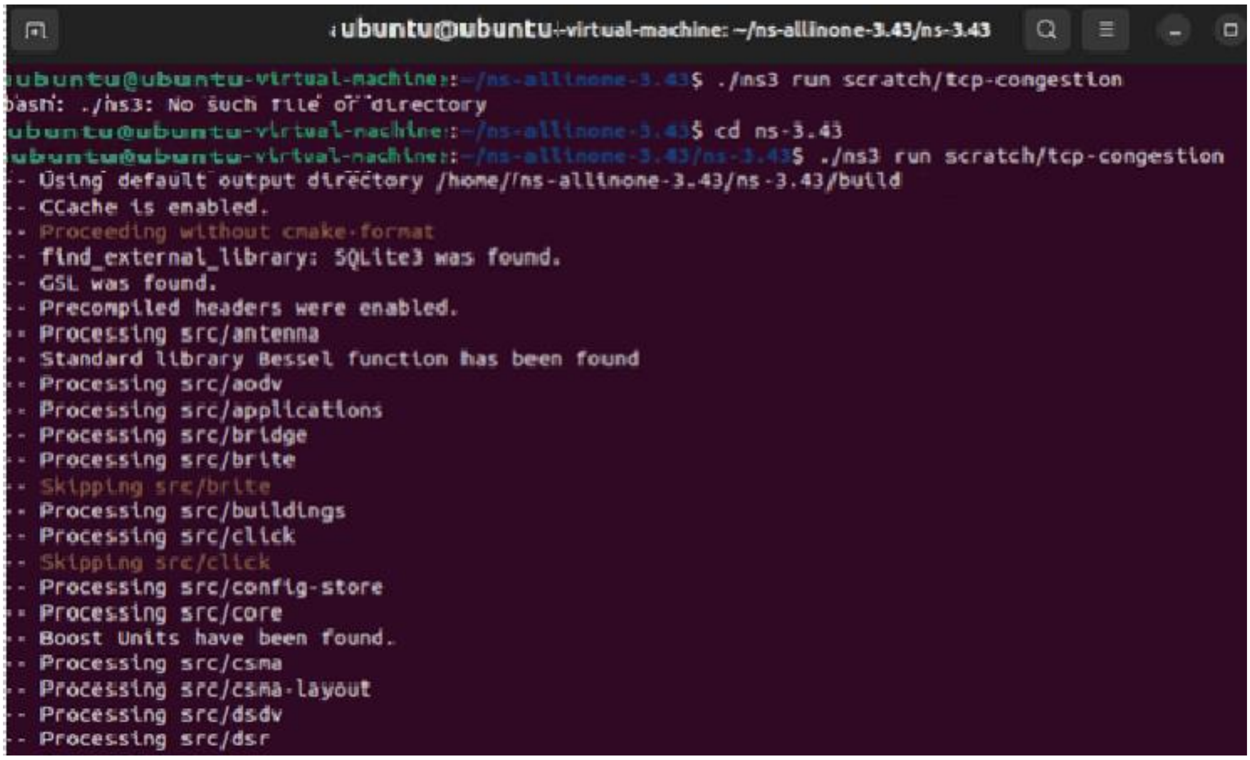
Laiba Fatima 22k-5195 Lab 7



Task1



ubuntu@ubuntu-virtual-machine: /ns-allinone-3.43/ns-3.43

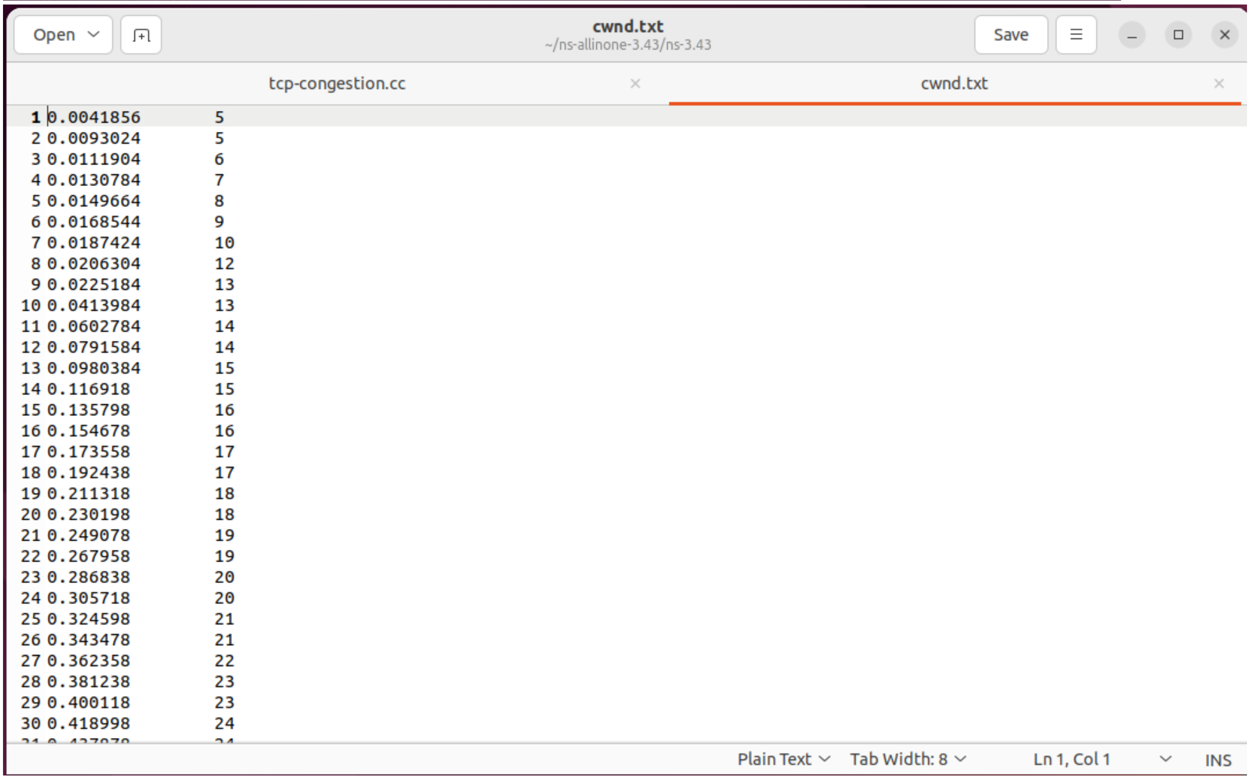
```
-- Processing src/core
-- Boost Units have been found.
-- Processing src/csma
-- Processing src/csma-layout
-- Processing src/dsdv
-- Processing src/dsr
-- Processing src/energy
-- Processing src/fd-net-device
-- Checking for module 'libdpdk'
--   No package 'libdpdk' found
-- Processing src/flow-monitor
-- Processing src/internet
-- Processing src/internet-apps
-- Processing src/lr-wpan
-- Processing src/lte
-- Processing src/mesh
-- Processing src/mobility
-- Processing src/netanim
-- Processing src/network
-- Processing src/nix-vector-routing
-- Processing src/olsr
-- Processing src/openflow
-- Skipping src/openflow
-- Processing src/point-to-point
-- Processing src/point-to-point-layout
-- Processing src/propagation
```

ubuntu@ubuntu-virtual-machine: ~/ns-allinone-3.43/ns-3.43

```
-- Processing src/wimax
-- ---- Summary of ns-3 settings:
Build profile                 : default
Build directory               : /home/ns-allinone-3.43/ns-3.43/build
Build with runtime asserts    : ON
Build with runtime logging    : ON
Build version embedding       : OFF (not requested)
BRITE Integration             : OFF (Missing headers: "Brite.h" and missing libraries: "brite")
DES Metrics event collection  : OFF (not requested)
DPDK NetDevice                : OFF (not requested)
Emulation FdNetDevice         : ON
Examples                      : ON
File descriptor NetDevice     : ON
GNU Scientific Library (GSL)  : ON
GtkConfigStore                : ON
LibXml2 support               : ON
MPI Support                   : OFF (not requested)
ns-3 Click Integration        : OFF (Missing headers: "simclick.h" and missing libraries: "nsclick cli
ck")
ns-3 OpenFlow Integration     : OFF (Missing headers: "openflow.h" and missing libraries: "openflow")
Netmap emulation FdNetDevice  : OFF (missing dependency)
PyViz visualizer              : OFF (Python Bindings are disabled)
Python Bindings               : OFF (not requested)
SQLite support                : ON
Eigen3 support                : ON
Tap Bridge                    : ON
Tap FdNetDevice               : ON
Tests                         : ON


Modules configured to be built:
antenna                  aodv                     applications
bridge                   buildings                config-store
core                     csma                     csma-layout
```

ubuntu@ubuntu-virtual-machine: ~/ns-allinone-3.43/ns-3.43

```
ython Bindings              : OFF (not requested)
QLite support               : ON
igen3 support               : ON
ap Bridge                   : ON
ap FdNetDevice              : ON
ests                        : ON


odules configured to be built:
ntenna              aodv                      applications
ridge               buildings                 config-store
ore                 csma                      csma-layout
sdv                 dsr                       energy
d-net-device        flow-monitor              internet
nternet-apps        lr-wpan                   lte
esh                 mobility                  netanim
etwork              nix-vector-routing        olsr
oint-to-point       point-to-point-layout     propagation
ixlowpan            spectrum                  stats
ap-bridge           test                      topology-read
raffic-control      uan                       virtual-net-device
ifi                 wimax

odules that cannot be built:
rite                click                     mpi
penflow             visualizer


- Configuring done
- Generating done
```

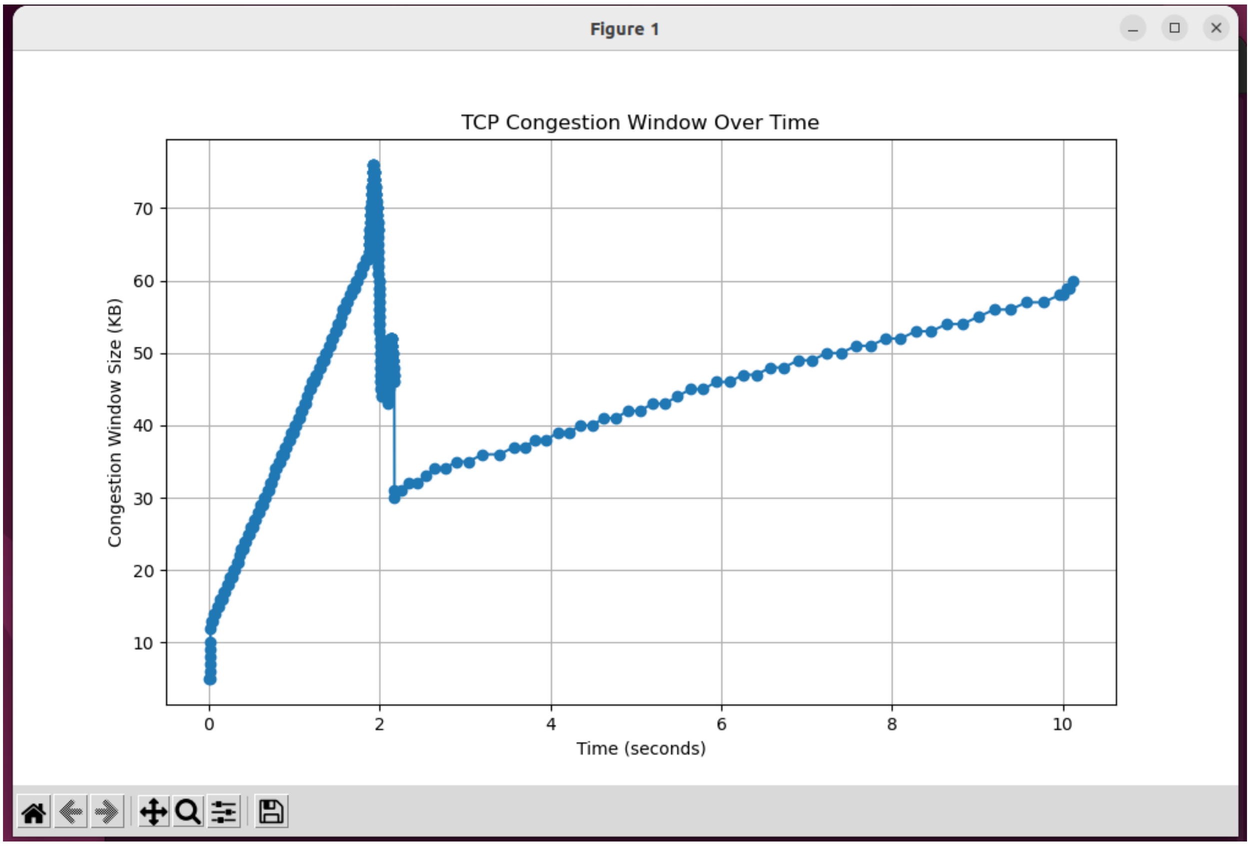


Open | cwnd.txt ~/ns-allinone-3.43/ns-3.43 | Save

tcp-congestion.cc | cwnd.txt

```
 1 0.0041856      5
 2 0.0093024      5
 3 0.0111904      6
 4 0.0130784      7
 5 0.0149664      8
 6 0.0168544      9
 7 0.0187424      10
 8 0.0206304      12
 9 0.0225184      13
10 0.0413984      13
11 0.0602784      14
12 0.0791584      14
13 0.0980384      15
14 0.116918       15
15 0.135798       16
16 0.154678       16
17 0.173558       17
18 0.192438       17
19 0.211318       18
20 0.230198       18
21 0.249078       19
22 0.267958       19
23 0.286838       20
24 0.305718       20
25 0.324598       21
26 0.343478       21
27 0.362358       22
28 0.381238       23
29 0.400118       23
30 0.418998       24
```

Plain Text | Tab Width: 8 | Ln 1, Col 1 | INS



Plot_cwnd.py

```
import matplotlib.pyplot as plt
# Lists to store time and congestion window size
time = []
cwnd = []
# Read the cwnd.txt file
with open("cwnd.txt", "r") as file:
    for line in file:
        parts = line.strip().split("\t")
        if len(parts) == 2:
            time.append(float(parts[0]))
            cwnd.append(float(parts[1]))
# Plotting
plt.figure(figsize=(10, 6))
plt.plot(time, cwnd, marker='o')
plt.title('TCP Congestion Window Over Time')
plt.xlabel('Time (seconds)')
plt.ylabel('Congestion Window Size (KB)')
plt.grid(True)
plt.show()
```


```python
import matplotlib.pyplot as plt

# Lists to store time and congestion window size
time = []
cwnd = []

# Read the cwnd.txt file
with open("cwnd.txt", "r") as file:
    for line in file:
        parts = line.strip().split("\t")
        if len(parts) == 2:
            time.append(float(parts[0]))
            cwnd.append(float(parts[1]))

# Plotting
plt.figure(figsize=(10, 6))
plt.plot(time, cwnd, marker='o')
plt.title('TCP Congestion Window Over Time')
plt.xlabel('Time (seconds)')
plt.ylabel('Congestion Window Size (KB)')
plt.grid(True)
plt.show()
```

Tcp-congestion.cc

```cpp
#include "ns3/core-module.h"
#include "ns3/network-module.h"
#include "ns3/internet-module.h"
#include "ns3/point-to-point-module.h"
#include "ns3/applications-module.h"
#include "ns3/flow-monitor-module.h"
using namespace ns3;
NS_LOG_COMPONENT_DEFINE ("TcpCongestionControlExample");
// This function writes (time, cwnd) to file
void
CwndChange (Ptr<OutputStreamWrapper> stream, uint32_t oldCwnd, uint32_t newCwnd)
{
  *stream->GetStream () << Simulator::Now ().GetSeconds () << "\t" << newCwnd / 1024 <<
std::endl;
}
// This function connects to the BulkSendApplication's socket *after* it starts
void
TraceCwnd (Ptr<Application> app, Ptr<OutputStreamWrapper> stream)
{
  Ptr<BulkSendApplication> bulkSendApp = DynamicCast<BulkSendApplication> (app);
  Ptr<Socket> sock = bulkSendApp->GetSocket ();
  if (sock)
  {
    sock->TraceConnectWithoutContext ("CongestionWindow", MakeBoundCallback
(&CwndChange, stream));
  }
}
```

```
int
main (int argc, char *argv[])
{
  Time::SetResolution (Time::NS);
  NodeContainer nodes;
  nodes.Create (2);
  PointToPointHelper pointToPoint;
  pointToPoint.SetDeviceAttribute ("DataRate", StringValue ("5Mbps"));
  pointToPoint.SetChannelAttribute ("Delay", StringValue ("2ms"));
  NetDeviceContainer devices;
  devices = pointToPoint.Install (nodes);
  InternetStackHelper stack;
  stack.Install (nodes);
  Ipv4AddressHelper address;
  address.SetBase ("10.1.1.0", "255.255.255.0");
  Ipv4InterfaceContainer interfaces = address.Assign (devices);
  uint16_t port = 9;  // Discard port
  BulkSendHelper source ("ns3::TcpSocketFactory",
                         InetSocketAddress (interfaces.GetAddress (1), port));
  source.SetAttribute ("MaxBytes", UintegerValue (0));
  ApplicationContainer sourceApps = source.Install (nodes.Get (0));
  sourceApps.Start (Seconds (0.0));
  sourceApps.Stop (Seconds (10.0));
  PacketSinkHelper sink ("ns3::TcpSocketFactory",
                         InetSocketAddress (Ipv4Address::GetAny (), port));
  ApplicationContainer sinkApps = sink.Install (nodes.Get (1));
  sinkApps.Start (Seconds (0.0));
  sinkApps.Stop (Seconds (10.0));
  // Enable Congestion Window Tracing
  AsciiTraceHelper asciiTraceHelper;
  Ptr<OutputStreamWrapper> stream = asciiTraceHelper.CreateFileStream ("cwnd.txt");
  // Schedule tracing after application starts
  Simulator::Schedule (Seconds (0.001), &TraceCwnd, sourceApps.Get(0), stream);
  Simulator::Run ();
  Simulator::Destroy ();
  return 0;
}
```

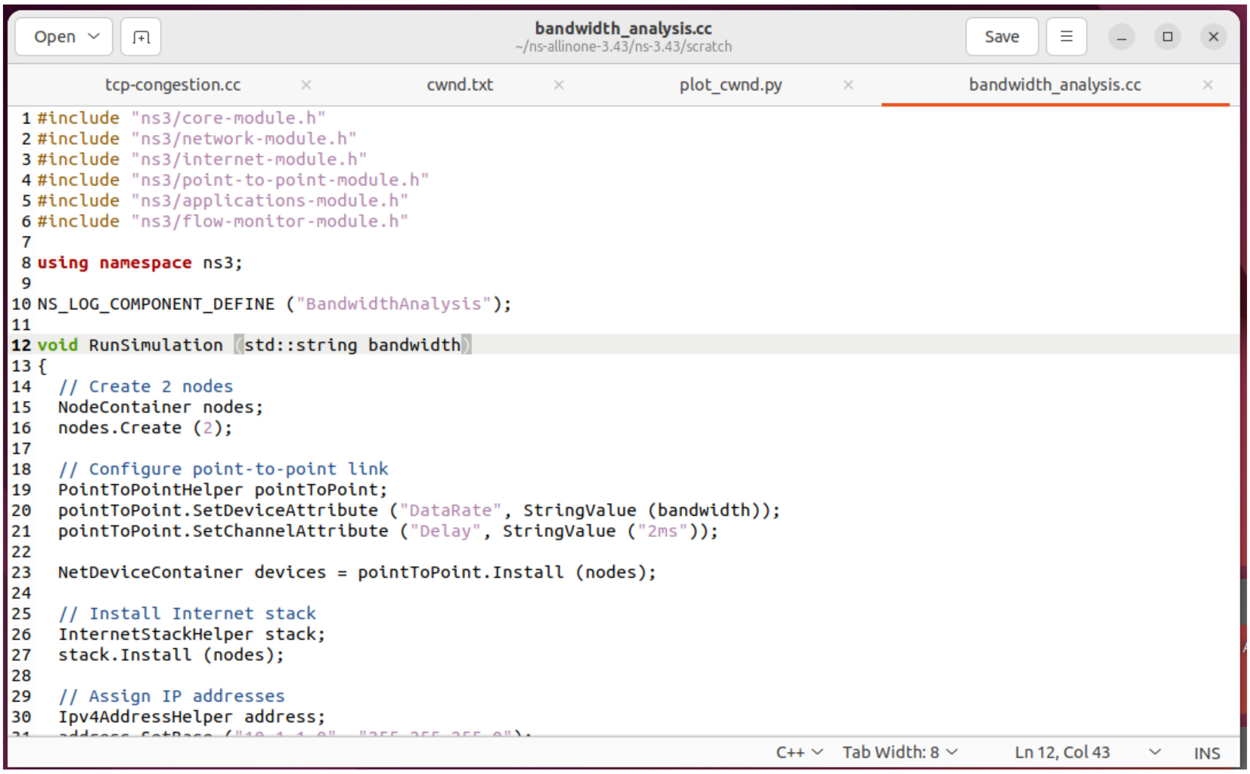
Task2

Bandwidth_analysis.cc

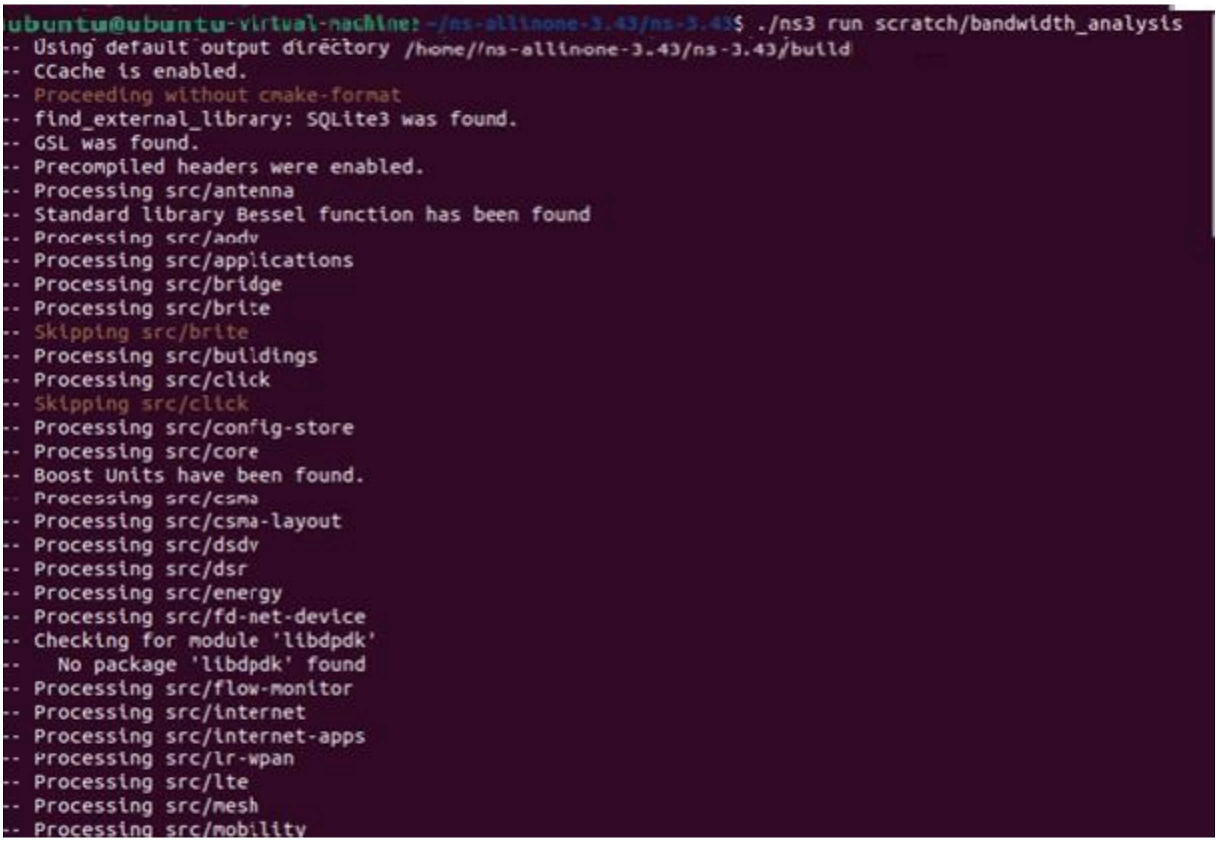
```
#include "ns3/core-module.h"
#include "ns3/network-module.h"
#include "ns3/internet-module.h"
#include "ns3/point-to-point-module.h"
#include "ns3/applications-module.h"
#include "ns3/flow-monitor-module.h"
```

```
using namespace ns3;

NS_LOG_COMPONENT_DEFINE ("BandwidthAnalysis");

void RunSimulation (std::string bandwidth)
{
  // Create 2 nodes
  NodeContainer nodes;
  nodes.Create (2);

  // Configure point-to-point link
  PointToPointHelper pointToPoint;
  pointToPoint.SetDeviceAttribute ("DataRate", StringValue (bandwidth));
  pointToPoint.SetChannelAttribute ("Delay", StringValue ("2ms"));

  NetDeviceContainer devices = pointToPoint.Install (nodes);

  // Install Internet stack
  InternetStackHelper stack;
  stack.Install (nodes);

  // Assign IP addresses
  Ipv4AddressHelper address;
  address.SetBase ("10.1.1.0", "255.255.255.0");
  Ipv4InterfaceContainer interfaces = address.Assign (devices);

  // Set up UDP traffic
  uint16_t port = 9;
  UdpServerHelper server (port);
  ApplicationContainer serverApp = server.Install (nodes.Get (1));
  serverApp.Start (Seconds (1.0));
  serverApp.Stop (Seconds (10.0));

  UdpClientHelper client (interfaces.GetAddress (1), port);
  client.SetAttribute ("MaxPackets", UintegerValue (32000));
  client.SetAttribute ("Interval", TimeValue (MilliSeconds (10))); // 10 ms between packets
  client.SetAttribute ("PacketSize", UintegerValue (1024));

  ApplicationContainer clientApp = client.Install (nodes.Get (0));
  clientApp.Start (Seconds (2.0));
  clientApp.Stop (Seconds (10.0));

  // Install FlowMonitor
  FlowMonitorHelper flowHelper;
  Ptr<FlowMonitor> monitor = flowHelper.InstallAll ();

  // Run Simulation
  Simulator::Stop (Seconds (11.0));
  Simulator::Run ();
```

```
  // Flow Monitor Results
  monitor->CheckForLostPackets ();
  Ptr<Ipv4FlowClassifier> classifier = DynamicCast<Ipv4FlowClassifier>
(flowHelper.GetClassifier ());
  std::map<FlowId, FlowMonitor::FlowStats> stats = monitor->GetFlowStats ();

  std::cout << "\nResults for Bandwidth = " << bandwidth << std::endl;
  for (auto& flow : stats)
    {
      Ipv4FlowClassifier::FiveTuple t = classifier->FindFlow (flow.first);
      if (t.destinationPort == port)
        {
          double throughput = flow.second.rxBytes * 8.0 /
(flow.second.timeLastRxPacket.GetSeconds () - flow.second.timeFirstTxPacket.GetSeconds ())
/ 1024 / 1024; // Mbps
          std::cout << "Flow ID: " << flow.first << ", Src Addr: " << t.sourceAddress << ", Dst Addr: "
<< t.destinationAddress << std::endl;
          std::cout << "  Throughput: " << throughput << " Mbps" << std::endl;
          std::cout << "  Packet Loss Ratio: " << (double)(flow.second.txPackets -
flow.second.rxPackets) / flow.second.txPackets << std::endl;
        }
    }

  Simulator::Destroy ();
}

int main (int argc, char *argv[])
{
  // Run for different bandwidths
  RunSimulation ("1Mbps");
  RunSimulation ("5Mbps");
  RunSimulation ("10Mbps");

  return 0;
}
```



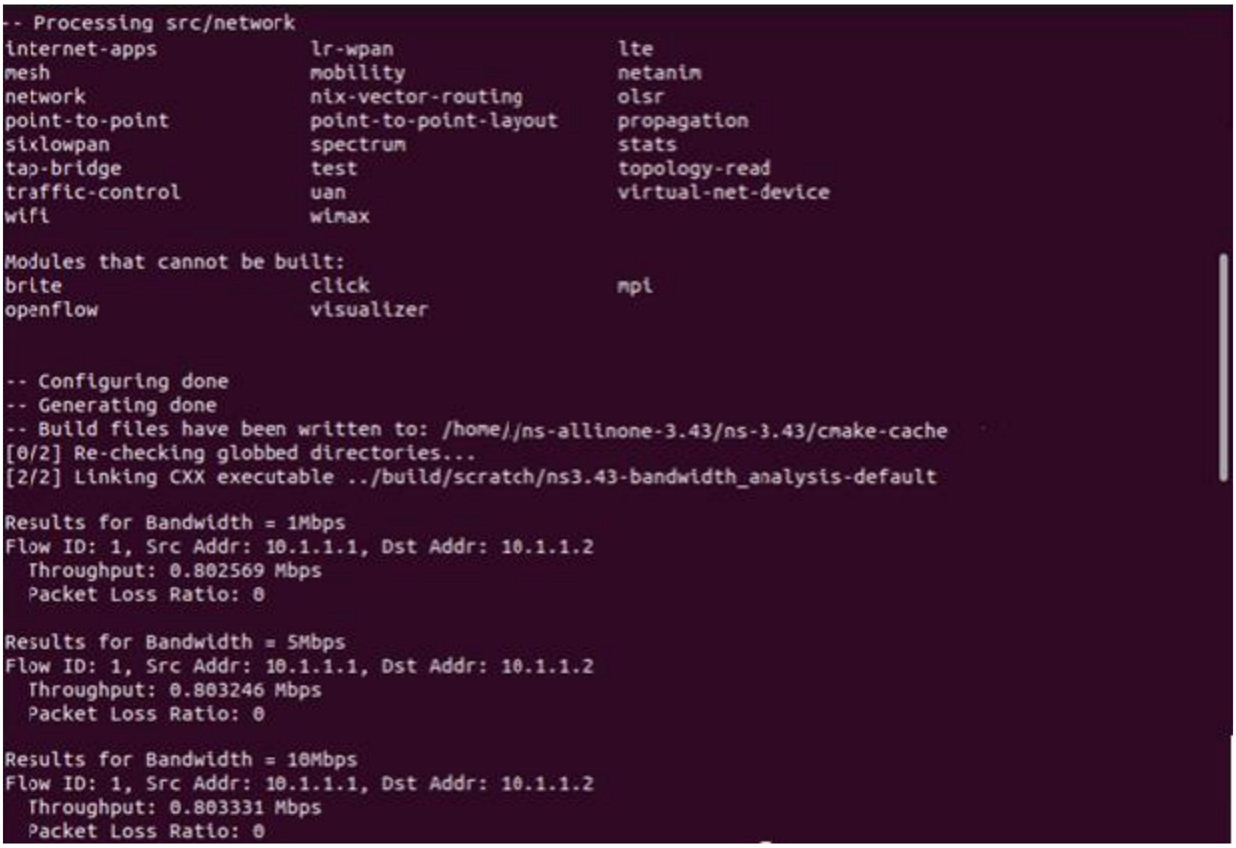

```cpp
#include "ns3/core-module.h"
#include "ns3/network-module.h"
#include "ns3/internet-module.h"
#include "ns3/point-to-point-module.h"
#include "ns3/applications-module.h"
#include "ns3/flow-monitor-module.h"

using namespace ns3;

NS_LOG_COMPONENT_DEFINE ("BandwidthAnalysis");

void RunSimulation (std::string bandwidth)
{
  // Create 2 nodes
  NodeContainer nodes;
  nodes.Create (2);

  // Configure point-to-point link
  PointToPointHelper pointToPoint;
  pointToPoint.SetDeviceAttribute ("DataRate", StringValue (bandwidth));
  pointToPoint.SetChannelAttribute ("Delay", StringValue ("2ms"));

  NetDeviceContainer devices = pointToPoint.Install (nodes);

  // Install Internet stack
  InternetStackHelper stack;
  stack.Install (nodes);

  // Assign IP addresses
  Ipv4AddressHelper address;
  address.SetBase ("10.1.1.0", "255.255.255.0");
```



```
ubuntu@ubuntu-virtual-machine:~/ns-allinone-3.43/ns-3.43$ ./ns3 run scratch/bandwidth_analysis
-- Using default output directory /home//ns-allinone-3.43/ns-3.43/build
-- CCache is enabled.
-- Proceeding without cmake-format
-- find_external_library: SQLite3 was found.
-- GSL was found.
-- Precompiled headers were enabled.
-- Processing src/antenna
-- Standard library Bessel function has been found
-- Processing src/aodv
-- Processing src/applications
-- Processing src/bridge
-- Processing src/brite
-- Skipping src/brite
-- Processing src/buildings
-- Processing src/click
-- Skipping src/click
-- Processing src/config-store
-- Processing src/core
-- Boost Units have been found.
-- Processing src/csma
-- Processing src/csma-layout
-- Processing src/dsdv
-- Processing src/dsr
-- Processing src/energy
-- Processing src/fd-net-device
-- Checking for module 'libdpdk'
--   No package 'libdpdk' found
-- Processing src/flow-monitor
-- Processing src/internet
-- Processing src/internet-apps
-- Processing src/lr-wpan
-- Processing src/lte
-- Processing src/mesh
-- Processing src/mobility
```

```
-- Processing src/network
-- Processing src/nix-vector-routing
-- Processing src/olsr
-- Processing src/openflow
-- Skipping src/openflow
-- Processing src/point-to-point
-- Processing src/point-to-point-layout
-- Processing src/propagation
-- Processing src/sixlowpan
-- Processing src/spectrum
-- Processing src/stats
-- Processing src/tap-bridge
-- Processing src/test
-- Processing src/topology-read
-- Processing src/traffic-control
-- Processing src/uan
-- Processing src/virtual-net-device
-- Processing src/wifi
-- Processing src/wimax
-- ---- Summary of ns-3 settings:
Build profile                 : default
Build directory               : /home/ns-allinone-3.43/ns-3.43/build
Build with runtime asserts    : ON
Build with runtime logging    : ON
Build version embedding       : OFF (not requested)
BRITE Integration             : OFF (Missing headers: "Brite.h" and missing libraries: "brite")
DES Metrics event collection  : OFF (not requested)
DPDK NetDevice                : OFF (not requested)
Emulation FdNetDevice         : ON
Examples                      : ON
File descriptor NetDevice     : ON
GNU Scientific Library (GSL)  : ON
GtkConfigStore                : ON
LibXml2 support               : ON
MPI Support                   : OFF (not requested)
```

```
GtkConfigStore                : ON
LibXml2 support               : ON
MPI Support                   : OFF (not requested)
ns-3 Click Integration        : OFF (Missing headers: "simclick.h" and missing libraries: "nsclick cli
ck")
ns-3 OpenFlow Integration     : OFF (Missing headers: "openflow.h" and missing libraries: "openflow")
Netmap emulation FdNetDevice  : OFF (missing dependency)
PyViz visualizer              : OFF (Python Bindings are disabled)
Python Bindings               : OFF (not requested)
SQLite support                : ON
Eigen3 support                : ON
Tap Bridge                    : ON
Tap FdNetDevice               : ON
Tests                         : ON


Modules configured to be built:
antenna                   aodv                      applications
bridge                    buildings                 config-store
core                      csma                      csma-layout
dsdv                      dsr                       energy
fd-net-device             flow-monitor              internet
internet-apps             lr-wpan                   lte
mesh                      mobility                  netanim
network                   nix-vector-routing        olsr
point-to-point            point-to-point-layout     propagation
sixlowpan                 spectrum                  stats
tap-bridge                test                      topology-read
traffic-control           uan                       virtual-net-device
wifi                      wimax

Modules that cannot be built:
brite                     click                     mpi
openflow                  visualizer
```



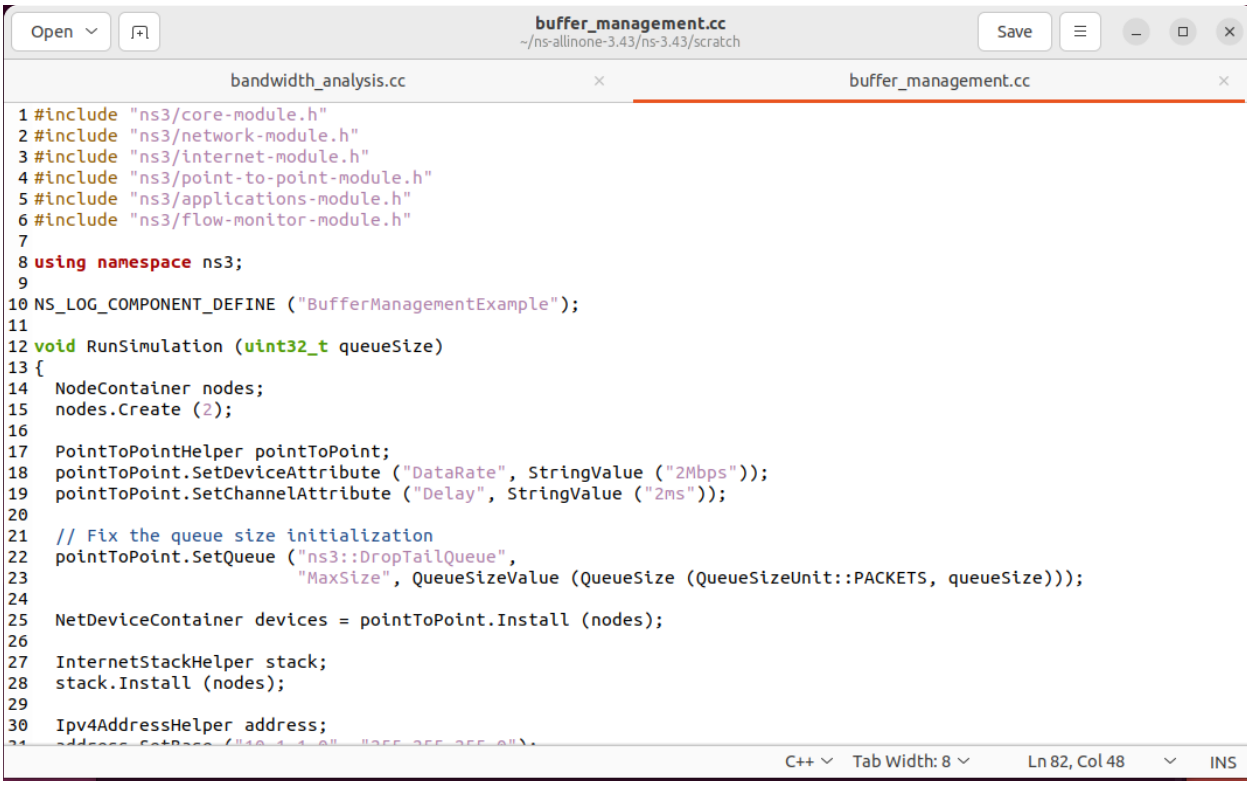
Task3

Buffer_management.cc

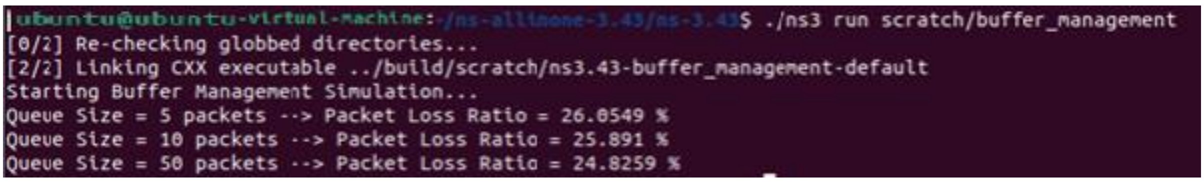
```
#include "ns3/core-module.h"
#include "ns3/network-module.h"
#include "ns3/internet-module.h"
#include "ns3/point-to-point-module.h"
#include "ns3/applications-module.h"
#include "ns3/flow-monitor-module.h"

using namespace ns3;

NS_LOG_COMPONENT_DEFINE ("BufferManagementExample");

void RunSimulation (uint32_t queueSize)
{
  NodeContainer nodes;
  nodes.Create (2);

  PointToPointHelper pointToPoint;
  pointToPoint.SetDeviceAttribute ("DataRate", StringValue ("2Mbps"));
  pointToPoint.SetChannelAttribute ("Delay", StringValue ("2ms"));

  pointToPoint.SetQueue ("ns3::DropTailQueue",
```

```
                       "MaxSize", QueueSizeValue (QueueSize (QueueSizeUnit::PACKETS,
queueSize)));

  NetDeviceContainer devices = pointToPoint.Install (nodes);

  InternetStackHelper stack;
  stack.Install (nodes);

  Ipv4AddressHelper address;
  address.SetBase ("10.1.1.0", "255.255.255.0");
  Ipv4InterfaceContainer interfaces = address.Assign (devices);

  uint16_t port = 9;
  OnOffHelper onoff ("ns3::UdpSocketFactory",
                     InetSocketAddress (interfaces.GetAddress (1), port));
  onoff.SetConstantRate (DataRate ("5Mbps")); // Higher than link capacity
  onoff.SetAttribute ("PacketSize", UintegerValue (1024));

  ApplicationContainer app = onoff.Install (nodes.Get (0));
  app.Start (Seconds (1.0));
  app.Stop (Seconds (5.0));

  PacketSinkHelper sink ("ns3::UdpSocketFactory",
                         InetSocketAddress (Ipv4Address::GetAny (), port));
  ApplicationContainer sinkApp = sink.Install (nodes.Get (1));
  sinkApp.Start (Seconds (0.0));
  sinkApp.Stop (Seconds (5.0));

  FlowMonitorHelper flowmon;
  Ptr<FlowMonitor> monitor = flowmon.InstallAll ();

  Simulator::Stop (Seconds (6.0));
  Simulator::Run ();

  monitor->CheckForLostPackets ();

  Ptr<Ipv4FlowClassifier> classifier = DynamicCast<Ipv4FlowClassifier> (flowmon.GetClassifier
());
  std::map<FlowId, FlowMonitor::FlowStats> stats = monitor->GetFlowStats ();

  for (auto it = stats.begin (); it != stats.end (); ++it)
    {
      Ipv4FlowClassifier::FiveTuple t = classifier->FindFlow (it->first);
      if (t.destinationPort == port)
        {
          double packetLossRatio = (double)(it->second.lostPackets) / (it->second.txPackets);
          std::cout << "Queue Size = " << queueSize
                    << " packets --> Packet Loss Ratio = "
                    << packetLossRatio * 100 << " %" << std::endl;
```

```
    }
  }

  Simulator::Destroy ();
}

int main (int argc, char *argv[])
{
  NS_LOG_UNCOND ("Starting Buffer Management Simulation...");

  RunSimulation (5);   // Queue Size 5 packets
  RunSimulation (10);  // Queue Size 10 packets
  RunSimulation (50);  // Queue Size 50 packets

  return 0;
}
```




```
#include "ns3/core-module.h"
#include "ns3/network-module.h"
#include "ns3/internet-module.h"
#include "ns3/point-to-point-module.h"
#include "ns3/applications-module.h"
#include "ns3/flow-monitor-module.h"

using namespace ns3;

NS_LOG_COMPONENT_DEFINE ("BufferManagementExample");

void RunSimulation (uint32_t queueSize)
{
  NodeContainer nodes;
  nodes.Create (2);

  PointToPointHelper pointToPoint;
  pointToPoint.SetDeviceAttribute ("DataRate", StringValue ("2Mbps"));
  pointToPoint.SetChannelAttribute ("Delay", StringValue ("2ms"));

  // Fix the queue size initialization
  pointToPoint.SetQueue ("ns3::DropTailQueue",
                         "MaxSize", QueueSizeValue (QueueSize (QueueSizeUnit::PACKETS, queueSize)));

  NetDeviceContainer devices = pointToPoint.Install (nodes);

  InternetStackHelper stack;
  stack.Install (nodes);

  Ipv4AddressHelper address;
```



```
ubuntu@ubuntu-virtual-machine:~/ns-allinone-3.43/ns-3.43$ ./ns3 run scratch/buffer_management
[0/2] Re-checking globbed directories...
[2/2] Linking CXX executable ../build/scratch/ns3.43-buffer_management-default
Starting Buffer Management Simulation...
Queue Size = 5 packets --> Packet Loss Ratio = 26.0549 %
Queue Size = 10 packets --> Packet Loss Ratio = 25.891 %
Queue Size = 50 packets --> Packet Loss Ratio = 24.8259 %
```

Task4

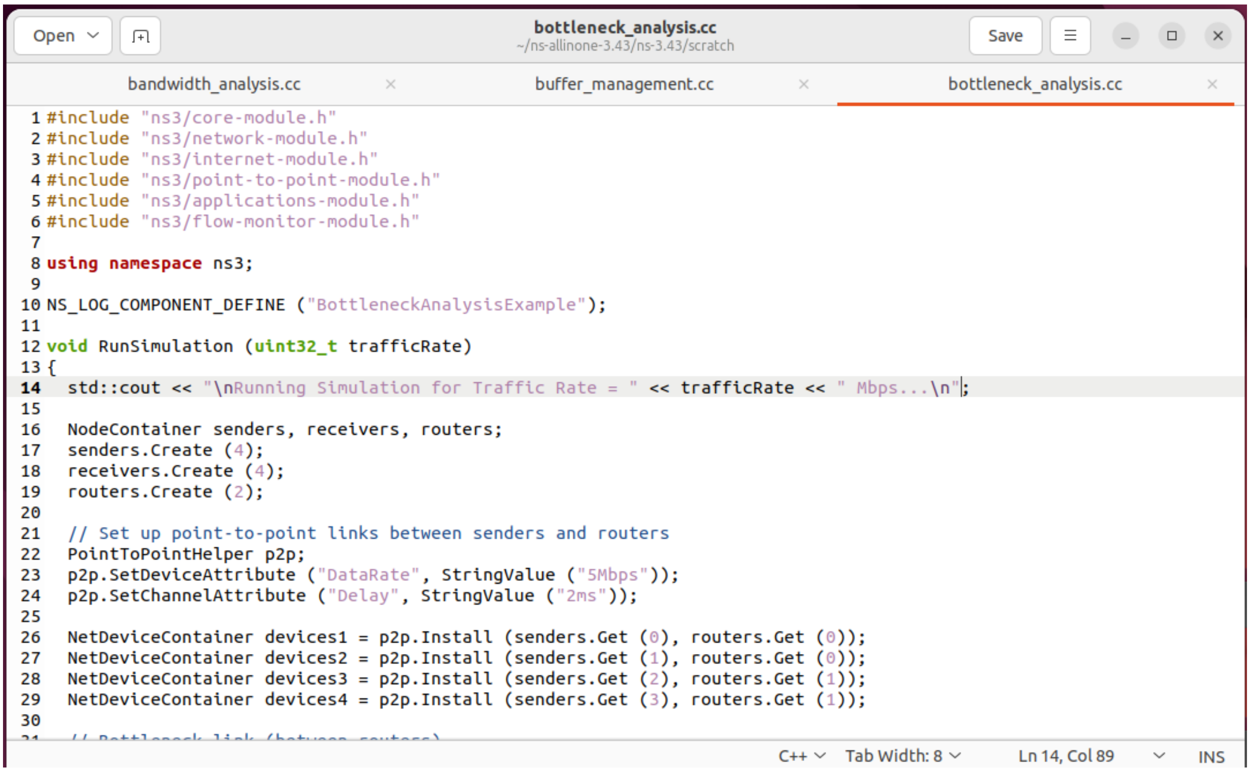
Bottleneck_analysis.cc

```
#include "ns3/core-module.h"
#include "ns3/network-module.h"
#include "ns3/internet-module.h"
#include "ns3/point-to-point-module.h"
#include "ns3/applications-module.h"
#include "ns3/flow-monitor-module.h"

using namespace ns3;

NS_LOG_COMPONENT_DEFINE ("BottleneckAnalysisExample");

void RunSimulation (uint32_t trafficRate)
{
  std::cout << "\nRunning Simulation for Traffic Rate = " << trafficRate << " Mbps...\n";

  NodeContainer senders, receivers, routers;
  senders.Create (4);
  receivers.Create (4);
  routers.Create (2);

  // Set up point-to-point links between senders and routers
  PointToPointHelper p2p;
  p2p.SetDeviceAttribute ("DataRate", StringValue ("5Mbps"));
  p2p.SetChannelAttribute ("Delay", StringValue ("2ms"));

  NetDeviceContainer devices1 = p2p.Install (senders.Get (0), routers.Get (0));
  NetDeviceContainer devices2 = p2p.Install (senders.Get (1), routers.Get (0));
  NetDeviceContainer devices3 = p2p.Install (senders.Get (2), routers.Get (1));
  NetDeviceContainer devices4 = p2p.Install (senders.Get (3), routers.Get (1));

  // Bottleneck link (between routers)
  PointToPointHelper bottleneck;
  bottleneck.SetDeviceAttribute ("DataRate", StringValue ("1Mbps"));
  bottleneck.SetChannelAttribute ("Delay", StringValue ("10ms"));

  NetDeviceContainer routerDevices = bottleneck.Install (routers.Get (0), routers.Get (1));

  // Links between routers and receivers
  NetDeviceContainer devices5 = p2p.Install (routers.Get (0), receivers.Get (0));
  NetDeviceContainer devices6 = p2p.Install (routers.Get (0), receivers.Get (1));
  NetDeviceContainer devices7 = p2p.Install (routers.Get (1), receivers.Get (2));
  NetDeviceContainer devices8 = p2p.Install (routers.Get (1), receivers.Get (3));

  // Install Internet Stack
  InternetStackHelper stack;
  stack.Install (senders);
  stack.Install (receivers);
  stack.Install (routers);
```

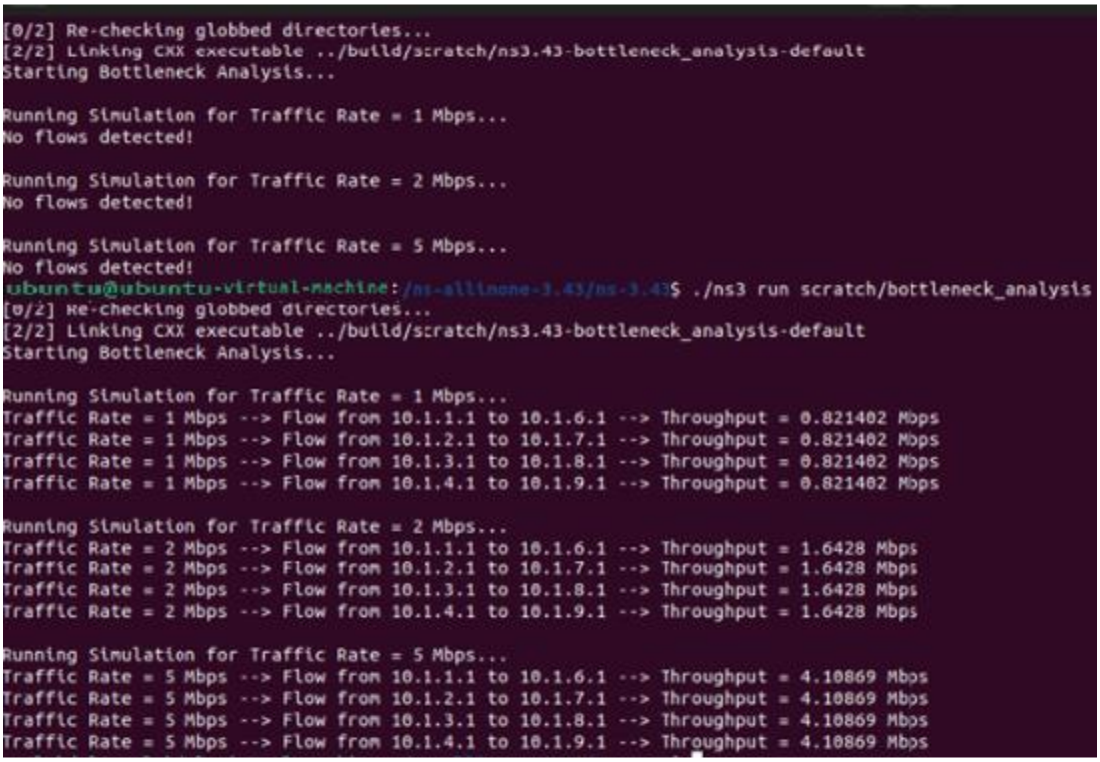
```
  // Assign IP addresses
  Ipv4AddressHelper address;
  address.SetBase ("10.1.1.0", "255.255.255.0");
  Ipv4InterfaceContainer senderInterfaces1 = address.Assign (devices1);

  address.SetBase ("10.1.2.0", "255.255.255.0");
  Ipv4InterfaceContainer senderInterfaces2 = address.Assign (devices2);

  address.SetBase ("10.1.3.0", "255.255.255.0");
  Ipv4InterfaceContainer senderInterfaces3 = address.Assign (devices3);

  address.SetBase ("10.1.4.0", "255.255.255.0");
  Ipv4InterfaceContainer senderInterfaces4 = address.Assign (devices4);

  address.SetBase ("10.1.5.0", "255.255.255.0");
  Ipv4InterfaceContainer routerInterfaces = address.Assign (routerDevices);

  address.SetBase ("10.1.6.0", "255.255.255.0");
  Ipv4InterfaceContainer receiverInterfaces1 = address.Assign (devices5);

  address.SetBase ("10.1.7.0", "255.255.255.0");
  Ipv4InterfaceContainer receiverInterfaces2 = address.Assign (devices6);

  address.SetBase ("10.1.8.0", "255.255.255.0");
  Ipv4InterfaceContainer receiverInterfaces3 = address.Assign (devices7);

  address.SetBase ("10.1.9.0", "255.255.255.0");
  Ipv4InterfaceContainer receiverInterfaces4 = address.Assign (devices8);

  Ipv4GlobalRoutingHelper::PopulateRoutingTables ();

  // UDP traffic generation
  uint16_t port = 9; // UDP port

  OnOffHelper onoff1 ("ns3::UdpSocketFactory", InetSocketAddress
(receiverInterfaces1.GetAddress (0), port));
  onoff1.SetConstantRate (DataRate (std::to_string (trafficRate) + "Mbps"));
  onoff1.SetAttribute ("PacketSize", UintegerValue (1024));

  OnOffHelper onoff2 ("ns3::UdpSocketFactory", InetSocketAddress
(receiverInterfaces2.GetAddress (0), port));
  onoff2.SetConstantRate (DataRate (std::to_string (trafficRate) + "Mbps"));
  onoff2.SetAttribute ("PacketSize", UintegerValue (1024));

  OnOffHelper onoff3 ("ns3::UdpSocketFactory", InetSocketAddress
(receiverInterfaces3.GetAddress (0), port));
  onoff3.SetConstantRate (DataRate (std::to_string (trafficRate) + "Mbps"));
  onoff3.SetAttribute ("PacketSize", UintegerValue (1024));
```

```
  OnOffHelper onoff4 ("ns3::UdpSocketFactory", InetSocketAddress
(receiverInterfaces4.GetAddress (0), port));
  onoff4.SetConstantRate (DataRate (std::to_string (trafficRate) + "Mbps"));
  onoff4.SetAttribute ("PacketSize", UintegerValue (1024));

  ApplicationContainer apps;
  apps.Add (onoff1.Install (senders.Get (0)));
  apps.Add (onoff2.Install (senders.Get (1)));
  apps.Add (onoff3.Install (senders.Get (2)));
  apps.Add (onoff4.Install (senders.Get (3)));

  apps.Start (Seconds (1.0));
  apps.Stop (Seconds (5.0));

  // Packet sinks at receivers
  PacketSinkHelper sink ("ns3::UdpSocketFactory", InetSocketAddress (Ipv4Address::GetAny
(), port));
  ApplicationContainer sinkApps;
  sinkApps.Add (sink.Install (receivers.Get (0)));
  sinkApps.Add (sink.Install (receivers.Get (1)));
  sinkApps.Add (sink.Install (receivers.Get (2)));
  sinkApps.Add (sink.Install (receivers.Get (3)));

  sinkApps.Start (Seconds (0.0));
  sinkApps.Stop (Seconds (6.0));

  // Flow Monitor
  FlowMonitorHelper flowmon;
  Ptr<FlowMonitor> monitor = flowmon.InstallAll ();

  Simulator::Stop (Seconds (6.0));
  Simulator::Run ();

  monitor->CheckForLostPackets ();

  Ptr<Ipv4FlowClassifier> classifier = DynamicCast<Ipv4FlowClassifier> (flowmon.GetClassifier
());
  std::map<FlowId, FlowMonitor::FlowStats> stats = monitor->GetFlowStats ();

  if (stats.size() == 0)
  {
    std::cout << "No flows detected!" << std::endl;
  }

  for (auto it = stats.begin (); it != stats.end (); ++it)
  {
    Ipv4FlowClassifier::FiveTuple t = classifier->FindFlow (it->first);
    if (t.destinationPort == port)
    {
```

```
      double throughput = (it->second.rxBytes * 8.0) / (5.0 * 1000000.0); // Convert to Mbps
      std::cout << "Traffic Rate = " << trafficRate
                << " Mbps --> Flow from " << t.sourceAddress
                << " to " << t.destinationAddress
                << " --> Throughput = " << throughput
                << " Mbps" << std::endl;
    }
  }

  Simulator::Destroy ();
}

int main (int argc, char *argv[])
{
  NS_LOG_UNCOND ("Starting Bottleneck Analysis...");

  RunSimulation (1);  // 1 Mbps
  RunSimulation (2);  // 2 Mbps
  RunSimulation (5);  // 5 Mbps

  return 0;
}
```




```
#include "ns3/core-module.h"
#include "ns3/network-module.h"
#include "ns3/internet-module.h"
#include "ns3/point-to-point-module.h"
#include "ns3/applications-module.h"
#include "ns3/flow-monitor-module.h"

using namespace ns3;

NS_LOG_COMPONENT_DEFINE ("BottleneckAnalysisExample");

void RunSimulation (uint32_t trafficRate)
{
  std::cout << "\nRunning Simulation for Traffic Rate = " << trafficRate << " Mbps...\n";

  NodeContainer senders, receivers, routers;
  senders.Create (4);
  receivers.Create (4);
  routers.Create (2);

  // Set up point-to-point links between senders and routers
  PointToPointHelper p2p;
  p2p.SetDeviceAttribute ("DataRate", StringValue ("5Mbps"));
  p2p.SetChannelAttribute ("Delay", StringValue ("2ms"));

  NetDeviceContainer devices1 = p2p.Install (senders.Get (0), routers.Get (0));
  NetDeviceContainer devices2 = p2p.Install (senders.Get (1), routers.Get (0));
  NetDeviceContainer devices3 = p2p.Install (senders.Get (2), routers.Get (1));
  NetDeviceContainer devices4 = p2p.Install (senders.Get (3), routers.Get (1));
```



```
[0/2] Re-checking globbed directories...
[2/2] Linking CXX executable ../build/scratch/ns3.43-bottleneck_analysis-default
Starting Bottleneck Analysis...

Running Simulation for Traffic Rate = 1 Mbps...
No flows detected!

Running Simulation for Traffic Rate = 2 Mbps...
No flows detected!

Running Simulation for Traffic Rate = 5 Mbps...
No flows detected!
ubuntu@ubuntu-virtual-machine:/ns-allinone-3.43/ns-3.43$ ./ns3 run scratch/bottleneck_analysis
[0/2] Re-checking globbed directories...
[2/2] Linking CXX executable ../build/scratch/ns3.43-bottleneck_analysis-default
Starting Bottleneck Analysis...

Running Simulation for Traffic Rate = 1 Mbps...
Traffic Rate = 1 Mbps --> Flow from 10.1.1.1 to 10.1.6.1 --> Throughput = 0.821402 Mbps
Traffic Rate = 1 Mbps --> Flow from 10.1.2.1 to 10.1.7.1 --> Throughput = 0.821402 Mbps
Traffic Rate = 1 Mbps --> Flow from 10.1.3.1 to 10.1.8.1 --> Throughput = 0.821402 Mbps
Traffic Rate = 1 Mbps --> Flow from 10.1.4.1 to 10.1.9.1 --> Throughput = 0.821402 Mbps

Running Simulation for Traffic Rate = 2 Mbps...
Traffic Rate = 2 Mbps --> Flow from 10.1.1.1 to 10.1.6.1 --> Throughput = 1.6428 Mbps
Traffic Rate = 2 Mbps --> Flow from 10.1.2.1 to 10.1.7.1 --> Throughput = 1.6428 Mbps
Traffic Rate = 2 Mbps --> Flow from 10.1.3.1 to 10.1.8.1 --> Throughput = 1.6428 Mbps
Traffic Rate = 2 Mbps --> Flow from 10.1.4.1 to 10.1.9.1 --> Throughput = 1.6428 Mbps

Running Simulation for Traffic Rate = 5 Mbps...
Traffic Rate = 5 Mbps --> Flow from 10.1.1.1 to 10.1.6.1 --> Throughput = 4.10869 Mbps
Traffic Rate = 5 Mbps --> Flow from 10.1.2.1 to 10.1.7.1 --> Throughput = 4.10869 Mbps
Traffic Rate = 5 Mbps --> Flow from 10.1.3.1 to 10.1.8.1 --> Throughput = 4.10869 Mbps
Traffic Rate = 5 Mbps --> Flow from 10.1.4.1 to 10.1.9.1 --> Throughput = 4.10869 Mbps
```

Task5

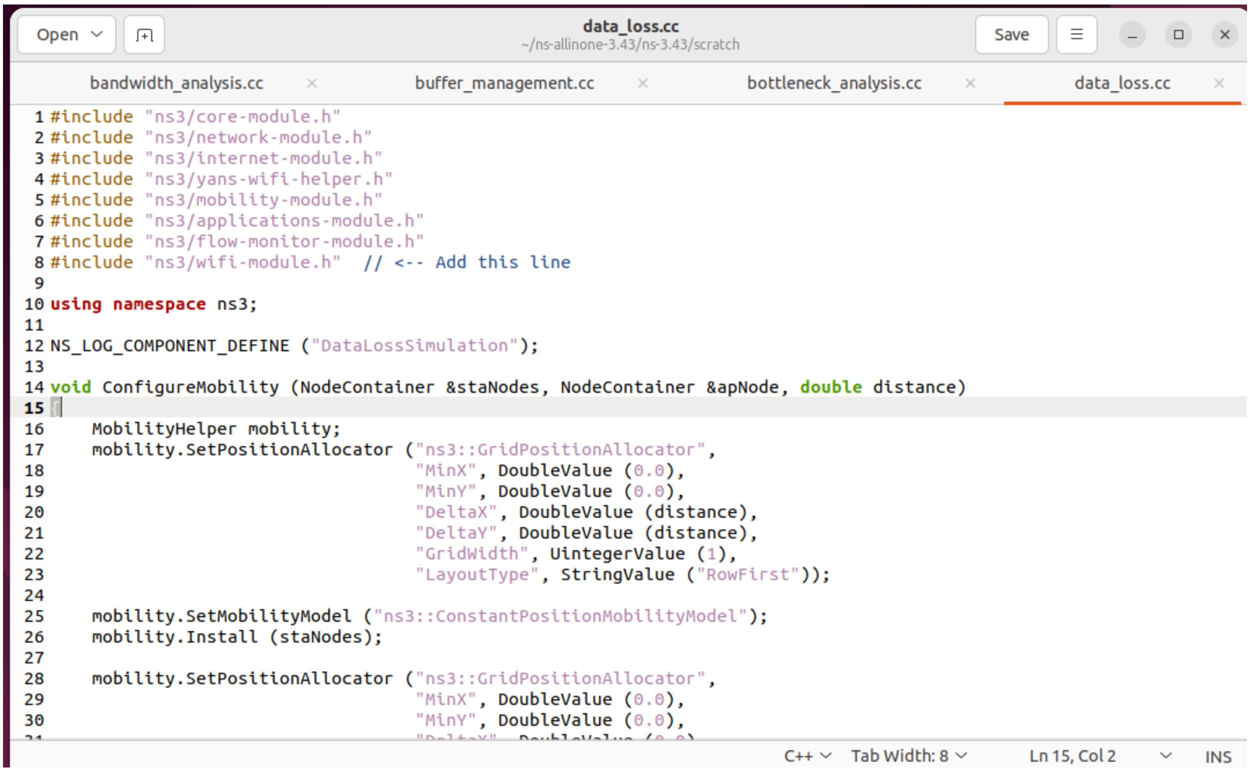
Data_loss.cc

```
#include "ns3/core-module.h"
#include "ns3/network-module.h"
#include "ns3/internet-module.h"
#include "ns3/yans-wifi-helper.h"
#include "ns3/mobility-module.h"
#include "ns3/applications-module.h"
#include "ns3/flow-monitor-module.h"
#include "ns3/wifi-module.h"  // <-- Add this line

using namespace ns3;

NS_LOG_COMPONENT_DEFINE ("DataLossSimulation");

void ConfigureMobility (NodeContainer &staNodes, NodeContainer &apNode, double distance)
{
  MobilityHelper mobility;
  mobility.SetPositionAllocator ("ns3::GridPositionAllocator",
                                 "MinX", DoubleValue (0.0),
                                 "MinY", DoubleValue (0.0),
                                 "DeltaX", DoubleValue (distance),
                                 "DeltaY", DoubleValue (distance),
                                 "GridWidth", UintegerValue (1),
                                 "LayoutType", StringValue ("RowFirst"));
```

```
  mobility.SetMobilityModel ("ns3::ConstantPositionMobilityModel");
  mobility.Install (staNodes);

  mobility.SetPositionAllocator ("ns3::GridPositionAllocator",
                                 "MinX", DoubleValue (0.0),
                                 "MinY", DoubleValue (0.0),
                                 "DeltaX", DoubleValue (0.0),
                                 "DeltaY", DoubleValue (0.0),
                                 "GridWidth", UintegerValue (1),
                                 "LayoutType", StringValue ("RowFirst"));
  mobility.Install (apNode);
}

int main (int argc, char *argv[])
{
  double distance = 10.0;
  CommandLine cmd;
  cmd.AddValue ("distance", "Distance between AP and STAs", distance);
  cmd.Parse (argc, argv);

  NodeContainer wifiStaNodes;
  wifiStaNodes.Create (3);
  NodeContainer wifiApNode;
  wifiApNode.Create (1);

  YansWifiChannelHelper channel = YansWifiChannelHelper::Default ();
  YansWifiPhyHelper phy;
  phy.SetChannel (channel.Create ());

  WifiHelper wifi;
  wifi.SetRemoteStationManager ("ns3::IdealWifiManager");


  WifiMacHelper mac;
  Ssid ssid = Ssid ("ns-3-ssid");

  mac.SetType ("ns3::StaWifiMac",
          "Ssid", SsidValue (ssid),
          "ActiveProbing", BooleanValue (false));
  NetDeviceContainer staDevices;
  staDevices = wifi.Install (phy, mac, wifiStaNodes);

  mac.SetType ("ns3::ApWifiMac",
          "Ssid", SsidValue (ssid));
  NetDeviceContainer apDevice;
  apDevice = wifi.Install (phy, mac, wifiApNode);

  InternetStackHelper stack;
  stack.Install (wifiApNode);
```

```
  stack.Install (wifiStaNodes);
  Ipv4AddressHelper address;
  address.SetBase ("10.1.3.0", "255.255.255.0");
  Ipv4InterfaceContainer staInterfaces;
  staInterfaces = address.Assign (staDevices);
  address.Assign (apDevice);
  // Mobility
  ConfigureMobility (wifiStaNodes, wifiApNode, distance);
  // UDP Traffic from AP to STAs
  uint16_t port = 9;
  UdpServerHelper server (port);
  ApplicationContainer serverApps;
  for (uint32_t i = 0; i < wifiStaNodes.GetN (); ++i)
  {
    serverApps.Add (server.Install (wifiStaNodes.Get (i)));
  }
  serverApps.Start (Seconds (1.0));
  serverApps.Stop (Seconds (10.0));
  UdpClientHelper client (staInterfaces.GetAddress (0), port);
  client.SetAttribute ("MaxPackets", UintegerValue (10000));
  client.SetAttribute ("Interval", TimeValue (Seconds (0.01)));
  client.SetAttribute ("PacketSize", UintegerValue (1024));
  ApplicationContainer clientApps;
  clientApps.Add (client.Install (wifiApNode.Get (0)));
  clientApps.Start (Seconds (2.0));
  clientApps.Stop (Seconds (10.0));
  // Interference traffic (Station-to-Station)
  UdpClientHelper interClient (staInterfaces.GetAddress (1), port);
  interClient.SetAttribute ("MaxPackets", UintegerValue (10000));
  interClient.SetAttribute ("Interval", TimeValue (Seconds (0.01)));
  interClient.SetAttribute ("PacketSize", UintegerValue (1024));
  ApplicationContainer interApps;
  interApps.Add (interClient.Install (wifiStaNodes.Get (2)));
  interApps.Start (Seconds (2.5));
  interApps.Stop (Seconds (10.0));
  // Flow Monitor
  FlowMonitorHelper flowmon;
  Ptr<FlowMonitor> monitor = flowmon.InstallAll ();
  Simulator::Stop (Seconds (11.0));
  Simulator::Run ();
  monitor->CheckForLostPackets ();
  Ptr<Ipv4FlowClassifier> classifier = DynamicCast<Ipv4FlowClassifier>
(flowmon.GetClassifier ());
  std::map<FlowId, FlowMonitor::FlowStats> stats = monitor->GetFlowStats ();

  for (auto iter = stats.begin (); iter != stats.end (); ++iter)
  {
    Ipv4FlowClassifier::FiveTuple t = classifier->FindFlow (iter->first);
```

```
        std::cout << "Flow " << iter->first << " (" << t.sourceAddress << " -> " <<
t.destinationAddress << ")\n";
        std::cout << "  Tx Packets: " << iter->second.txPackets << "\n";
        std::cout << "  Rx Packets: " << iter->second.rxPackets << "\n";
        std::cout << "  Lost Packets: " << (iter->second.txPackets - iter->second.rxPackets) <<
"\n";
        std::cout << "  Throughput: " << (iter->second.rxBytes * 8.0 / (iter-
>second.timeLastRxPacket.GetSeconds() - iter->second.timeFirstTxPacket.GetSeconds())) /
1024 / 1024  << " Mbps\n";
    }
    Simulator::Destroy ();
    return 0;
}
```



```
[2/2] Linking CXX executable ../build/scratch/ns3.43-data_loss-default
Flow 1 (10.1.3.4 -> 10.1.3.1)
  Tx Packets: 800
  Rx Packets: 800
  Lost Packets: 0
  Throughput: 0.803609 Mbps
Flow 2 (10.1.3.3 -> 10.1.3.2)
  Tx Packets: 750
  Rx Packets: 750
  Lost Packets: 0
  Throughput: 0.803635 Mbps
ubuntu@ubuntu-virtual-machine:~/ns-allinone-3.43/ns-3.43$ ./ns3 run "scratch/data_loss --distance=50"
[0/2] Re-checking globbed directories...
ninja: no work to do.
Flow 1 (10.1.3.4 -> 10.1.3.1)
  Tx Packets: 800
  Rx Packets: 800
  Lost Packets: 0
  Throughput: 0.803609 Mbps
Flow 2 (10.1.3.3 -> 10.1.3.2)
  Tx Packets: 750
  Rx Packets: 0
  Lost Packets: 750
  Throughput: -0 Mbps
ubuntu@ubuntu-virtual-machine:~/ns-allinone-3.43/ns-3.43$ ./ns3 run "scratch/data_loss --distance=100"
[0/2] Re-checking globbed directories...
ninja: no work to do.
Flow 1 (10.1.3.4 -> 10.1.3.1)
  Tx Packets: 800
  Rx Packets: 800
  Lost Packets: 0
  Throughput: 0.803609 Mbps
Flow 2 (10.1.3.3 -> 10.1.3.2)
  Tx Packets: 750
  Rx Packets: 0
  Lost Packets: 750
  Throughput: -0 Mbps
```

Flow1: Without Interference

Flow2: With Interference.